FitForge — Phase 3: Authentication, JWT, Refresh Tokens & Authorization

Status: **complete and verified on 2026-10-29.** Build: 0 warnings, 0 errors. 38 scripted HTTP checks plus 4 password-reset checks passed against the running API. Automated xUnit tests come in Phase 9.

1. What was built

Area	Result
Endpoints	`register`, `login`, `refresh`, `logout`, `change-password`, `forgot-password`, `reset-password`, `me` under `/api/auth`
Passwords	ASP.NET Core Identity hasher (PBKDF2-HMAC-SHA512, 100,000 iterations, per-user salt)
Access token	JWT, HMAC-SHA256, 15 minutes, claims `sub`, `email`, `role`, `jti`, `iss`, `aud`
Refresh token	64 random bytes, HttpOnly cookie, 7 days, **rotated on every use**, stored as SHA-256 hash, reuse revokes the session
Authorization	Fallback policy (every endpoint needs a token unless `[AllowAnonymous]`), role policies `AdminOnly`, `TrainerOrAdmin`, `MemberOnly`
Validation	FluentValidation validators, run by a global action filter, camelCase error keys
Errors	Global exception handler → RFC 7807 ProblemDetails; no stack traces leave the server
Brute force	Identity lockout: 5 wrong passwords → 15 minutes (per-IP rate limits come in Phase 5)

2. Packages added

Project	Package	Version
Infrastructure	Microsoft.AspNetCore.Authentication.JwtBearer	10.0.14
Infrastructure	FrameworkReference `Microsoft.AspNetCore.App`	(shared framework) — Identity token providers, Data Protection
Application	FluentValidation	12.2.0
Application	FluentValidation.DependencyInjectionExtensions	12.2.0

3. Files

```
src/GymManagement.Application/
├─ Common/Exceptions/AppExceptions.cs     AppException + BusinessRule(400),
AuthenticationFailed(401),
│                                         Forbidden(403), NotFound(404), Conflict(409),
RequestValidation(400)
├─ DTOs/Auth/AuthDtos.cs
Register/Login/ChangePassword/ForgotPassword/ResetPassword requests,
│                                         CurrentUserDto, AuthResponse, AuthResult
├─ Interfaces/IAuthService.cs
├─ Interfaces/ICurrentUser.cs             ICurrentUser, IEmailSender
├─ Validators/AuthValidators.cs           one validator per request + shared StrongPassword rule
└─ DependencyInjection.cs                 AddApplication(): validators, camelCase error keys

src/GymManagement.Infrastructure/
├─ Identity/JwtOptions.cs                 JwtOptions, FrontendOptions
├─ Identity/JwtTokenGenerator.cs          creates access tokens
├─ Identity/RefreshTokenService.cs        create / find / revoke family / revoke all
├─ Services/AuthService.cs                all auth flows
├─ Services/LoggingEmailSender.cs         dev-only email sender (writes to log)
└─ DependencyInjection.cs                 + token providers, Data Protection, JwtOptions
validation, services

src/GymManagement.Api/
├─ Authorization/Policies.cs              fallback + role policies
├─ Authorization/CurrentUser.cs           ICurrentUser from the validated JWT
├─ Extensions/AuthenticationExtensions.cs JWT bearer validation parameters
├─ Filters/ValidationFilter.cs            runs FluentValidation before every action
├─ Filters/RequireClientHeaderAttribute.cs CSRF header check for cookie endpoints
├─ Middleware/GlobalExceptionHandler.cs   exceptions → ProblemDetails
├─ Controllers/AuthController.cs
├─ Program.cs                             wiring + pipeline order
└─ appsettings.json                       + Jwt (issuer, audience, lifetimes), Frontend:BaseUrl
```

4. Configuration and secrets

`appsettings.json` (committed, no secrets):

```
"Jwt": { "Issuer": "FitForge.Api", "Audience": "FitForge.Client", "AccessTokenMinutes": 15,
"RefreshTokenDays": 7 },
"Frontend": { "BaseUrl": "http://localhost:5173" }
```

The signing key is a secret (user secrets locally, `Jwt__SigningKey` environment variable on a server):

```
# Generate a random 64-byte key and store it
$bytes = New-Object byte[] 64;
[Security.Cryptography.RandomNumberGenerator]::Create().GetBytes($bytes)
dotnet user-secrets set "Jwt:SigningKey" ([Convert]::ToBase64String($bytes)) --project
src/GymManagement.Api
```

If the key is missing or shorter than 32 bytes, the app **refuses to start** with: `Jwt:SigningKey is missing or shorter than 32 bytes. Set it with 'dotnet user-secrets set Jwt:SigningKey <value>'.`

5. How it works

Request pipeline (order matters)

```
UseExceptionHandler → (HSTS + HTTPS redirect outside Development) → UseAuthentication →
UseAuthorization → controllers
                                                                                    ⌞

ValidationFilter runs before each action
```

Login

1. Find user by email. **Unknown email:** verify the password against a dummy hash anyway (same timing), return 401.
2. Locked out → 401. Wrong password → record failure (5 = lockout), 401. Deactivated → 401. Every failure returns the same text: `Invalid email or password.`
3. Success → reset failure count, issue access token + new refresh token in a **new session family**.

Refresh-token rotation and reuse detection

```
login ─► T1 (family F)
refresh with T1 ─► T1 revoked, T1.ReplacedBy = hash(T2), cookie = T2
refresh with T2 ─► T2 revoked, cookie = T3 ...

refresh with T1 again ─► T1 already replaced ⇒ someone copied it
                         ⇒ revoke every token in family F (T2/T3 stop working too) ⇒ 401
```

Tokens revoked by logout or password change are treated as a normal session end (401, no security warning). Only reuse of a *rotated* token is logged as an attack.

Cookie settings

Setting	Value	Why
Name	`fitforge_refresh`	
HttpOnly	true	JavaScript (and any XSS) cannot read it
Secure	true	HTTPS only (browsers also accept it on `http://localhost`)
SameSite	Strict	never sent on requests started by another site
Path	`/api/auth`	not sent with ordinary API calls
Expires	7 days	

CSRF protection

Only `/api/auth/refresh` and `/api/auth/logout` use the cookie. Both require the header `X-FitForge-Client: web`, which a cross-site form can't send; a cross-site `fetch` with that header needs a CORS preflight, which Phase 5's CORS policy will reject. Missing header → **403**.

Token storage in the browser (implemented in Phase 6)

- Access token: kept in a JavaScript variable (memory). Lost on reload → the app calls `/refresh` once at startup.

- Never in `localStorage` / `sessionStorage`: an XSS bug could read those.
- Trade-off: an XSS bug could still *use* the in-memory token while the page is open, so XSS prevention (React's escaping, no `dangerouslySetInnerHTML`, a Content-Security-Policy in Phase 5) still matters.

Authorization

- **Fallback policy:** endpoints without `[AllowAnonymous]` require a valid token. Forgetting `[Authorize]` on a new endpoint is safe.
- **Role policies:** `[Authorize(Policy = Policies.AdminOnly)]` etc. Used by the Phase 4 controllers.
- **Resource-level checks** (member can only see their own records, trainer only their members) are done in services in Phase 4, using `ICurrentUser.UserId` from the token, never an ID from the URL.

Password change and reset

- **Change password:** wrong current password → 400 on `currentPassword`. Success → every refresh token of the user is revoked (all devices signed out) and the cookie is deleted.
- **Forgot password:** always 202 with the same message, whether or not the account exists. If it exists, Identity creates a **single-use reset token valid for 30 minutes**; the link `http://localhost:5173/reset-password?email=…&token=…` is "emailed" (logged in development).
- **Reset password:** invalid/expired/reused token → 400 `This password reset link is invalid or has expired.` Success → clears lockout and signs out every device.

Error format (ProblemDetails)

```
{
  "type": "https://tools.ietf.org/html/rfc9110#section-15.5.1",
  "title": "Validation failed",
  "status": 400,
  "detail": "One or more validation errors occurred.",
  "errors": { "password": ["Password must contain an uppercase letter.", "Password must contain a
digit.", "Password must contain a symbol."] },
  "traceId": "00-…"
}
```

---

6. API reference

Method	Route	Auth	Success	Errors
POST	`/api/auth/register`	anonymous	201 + `AuthResponse`, sets cookie	400 validation, 409 email exists
POST	`/api/auth/login`	anonymous	200 + `AuthResponse`, sets cookie	400, 401
POST	`/api/auth/refresh`	cookie + `X-FitForge-Client`	200 + `AuthResponse`, rotates cookie	401, 403 (no header)
POST	`/api/auth/logout`	cookie + `X-FitForge-Client`	204, deletes cookie	403 (no header)
POST	`/api/auth/change-password`	Bearer	204, deletes cookie	400, 401

Method	Route	Auth	Success	Errors
POST	`/api/auth/forgot-password`	anonymous	202 (always)	400 (bad email format)
POST	`/api/auth/reset-password`	anonymous	204	400
GET	`/api/auth/me`	Bearer	200 + `CurrentUserDto`	401

`AuthResponse`:

```
{
  "accessToken": "eyJhbGciOiJIUzI1NiIs…",
  "accessTokenExpiresUtc": "2026-10-29T06:30:51Z",
  "user": {
    "id": "6a73ed5f-c768-45ff-8a2a-08df35857a47",
    "email": "rahul.member@fitforge.test",
    "firstName": "Rahul", "lastName": "Sharma",
    "role": "Member", "memberProfileId": 1, "trainerProfileId": null
  }
}
```

---

7. Verification (run against the live API)

Script: 38 checks, all passed.

Group	Checks
Registration	201; **role forced to Member even when the body says `"role":"Admin"`**; cookie is HttpOnly; refresh token not in body; duplicate email in different case → 409; weak password → 400 with camelCase `password` key
Login	correct → 200; wrong password → 401; unknown email → 401 with **identical** message; seeded admin/trainer log in with correct role and profile ID
JWT	contains exactly `sub, email, role, jti, iss, aud` (+ `exp, iat, nbf`); lifetime exactly 900 s
Protected endpoints	`/me` with token 200, without 401, **tampered signature 401**; unknown `/api/*` route 401 (fallback policy); `/health` public
Refresh	no CSRF header → 403; with header → 200 and **cookie rotated**; reusing the old token → 401 **and the new token is revoked too**; no cookie → 401
Logout	204; refresh afterwards → 401
Change password	wrong current → 400; success → 204; old refresh token revoked; old password fails; new password works
Forgot password	existing and unknown email → **byte-identical** 202 responses
Lockout	after 5 wrong passwords, the **correct** password is refused

Password reset (token taken from the development log):

Attempt	Result
Tampered token	400 "invalid or has expired"

Attempt	Result
Real token	204
Same token again	400 (single-use)
Login with new password while previously locked out	200 (reset cleared the lockout)

Database and log checks:

- `RefreshTokens.TokenHash` is always 64 hex characters (SHA-256); raw tokens are never stored.
- The server log contained **no passwords and no JWTs**.
- One bug found and fixed during verification: tokens revoked by logout/password change were logged as "reuse detected". Now only reuse of an already-rotated token raises that warning.

Try it yourself (PowerShell)

```
dotnet run --project src/GymManagement.Api --launch-profile http

# In another terminal
$body = @{ email = "rahul.member@fitforge.test"; password = "<demo password>" } | ConvertTo-Json
$login = Invoke-RestMethod http://localhost:5080/api/auth/login -Method Post -ContentType
"application/json" -Body $body
Invoke-RestMethod http://localhost:5080/api/auth/me -Headers @{ Authorization = "Bearer
$($login.accessToken)" }
```

Or open `src/GymManagement.Api/GymManagement.Api.http` in Visual Studio / VS Code (REST Client).

---

8. Common mistakes

- **Putting the signing key in appsettings.json.** Anyone with the repo could forge admin tokens. Use user secrets / environment variables.
- `MapInboundClaims` **left on.** .NET then renames `sub` and `role` to long URIs, and `[Authorize(Roles = "Admin")]` silently fails. It is turned off here.
- **Calling** `/refresh` **from two browser tabs at once.** Both send the same token; the second looks like reuse and ends the session. The Phase 6 API client allows only one refresh request at a time.
- **Testing** `/refresh` **in Postman/Swagger without the** `X-FitForge-Client: web` **header** → 403 by design.
- **Expecting deactivation to cut off an access token immediately.** Deactivation blocks login and refresh at once; an already-issued access token remains valid until it expires (max 15 minutes). This is the standard trade-off of short-lived JWTs.
- **Registering reveals whether an email exists (409).** Unavoidable for a sign-up form; Phase 5 rate-limits `/register` to make bulk probing impractical. Login and forgot-password do not reveal it.

9. Security summary

Threat	Mitigation
Password database leak	PBKDF2-SHA512, 100k iterations, per-user salt
Refresh-token database leak	Only SHA-256 hashes stored
Stolen refresh token	Rotation + reuse detection revokes the whole session

Threat	Mitigation
XSS reading tokens	Refresh token HttpOnly; access token in memory only, 15 min
CSRF on cookie endpoints	SameSite=Strict + required custom header (+ CORS in Phase 5)
Brute-force login	Account lockout now; per-IP rate limit in Phase 5
Account enumeration	Generic login errors, dummy-hash timing, identical forgot-password responses
Privilege escalation at sign-up	No role in RegisterRequest; Member always assigned (verified)
Forged/altered JWT	Signature, issuer, audience, lifetime and algorithm all validated (tamper test → 401)
Error details leaking	ProblemDetails with safe messages; 500s show only a traceId

Phase checklist

- [x] Phase 1 — Architecture, requirements, data model, folder structure, dependencies
- [x] Phase 2 — Solution setup, entities, DbContext, configurations, migrations, seed data
- [x] Phase 3 — Identity, password hashing, JWT, refresh tokens, roles, policies
- [ ] Phase 4 — Services, DTOs, validation, CRUD APIs, resource-level authorization
- [ ] Phase 5 — Rate limiting, CORS, exception handling, audit logging, Swagger, hardening
- [ ] Phase 6 — React setup, design system, routing, auth integration, layouts
- [ ] Phase 7 — Admin, trainer, member dashboards on live APIs
- [ ] Phase 8 — Memberships, payments, workouts, diets, bookings, attendance UI
- [ ] Phase 9 — Automated tests, fixes, responsive and security checks
- [ ] Phase 10 — README, demo prep, resume description